深度学习基础

第 5 讲 — PyTorch 深度学习编程实战

孟翰

人工智能学院
中国石油大学（北京）

2026 年春季学期

本讲内容

1. 回顾与本讲目标
2. Tensor：PyTorch 的核心数据结构
3. 自动求导：autograd
4. nn.Module：构建神经网络
5. 数据管道：Dataset 与 DataLoader
6. 完整项目实战：FashionMNIST 分类
7. 常见 Bug 与调试技巧
8. 模型保存、加载与实验管理
9. 完整代码汇总与总结

第 1 讲 深度学习导论	第 2 讲 数学基础	第 3 讲 感知机到 MLP	第 4 讲 训练技巧
AI/ML/DL 关系	线代/微积分/概率	反向传播四公式	初始化/BN/正则化/Adam

你现在知道了这些

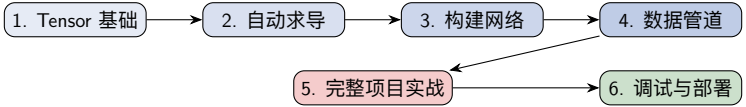
- 前向传播：$\boldsymbol{a}^{(l)} = f(\boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)})$
- 反向传播：$\boldsymbol{\delta}^{(l)} = (\boldsymbol{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)} \odot f'(\boldsymbol{z}^{(l)})$
- 标准配置：He 初始化 + BN + ReLU + Dropout + Adam + 交叉熵

但是……

"知道原理" $\neq$ "能写出代码"。**你能独立用 PyTorch 训练一个模型吗?**

一句话目标

从**零开始**，用 PyTorch 搭建一个完整的 FashionMNIST 分类项目，跑通整个流程。



今天覆盖的完整流程

1. PyTorch 张量操作与 GPU 加速
2. `autograd` 自动求导机制
3. `nn.Module` 搭建网络
4. `Dataset/DataLoader` 数据管道
5. **FashionMNIST 完整项目**：从数据加载到模型评估
6. 训练监控、常见 Bug 排查、模型保存

PyTorch vs TensorFlow

	PyTorch	TensorFlow
计算图	动态	静态/动态
调试	**像 Python**	较复杂
学术论文	**>70%**	~25%
工业部署	日趋成熟	成熟
学习曲线	**平缓**	较陡

PyTorch 的核心优势

- **动态图**：像写普通 Python 一样
- **直觉性**：与 NumPy 高度一致
- **调试友好**：可以用 print、断点
- **生态丰富**：torchvision 等
- **学术主流**：论文几乎都用

安装

`pip install torch torchvision` 或 `conda install pytorch torchvision -c pytorch`

Tensor = 多维数组 + GPU 加速 + 自动求导

PyTorch 中所有数据都以 Tensor 存储和计算。它和 NumPy 的 `ndarray` 非常相似，但多了两个关键能力：

= PyTorch Tensor

不同维度的 Tensor

维度	名称	示例	用途
0 维	标量	`torch.tensor(3.14)`	损失值
1 维	向量	`torch.tensor([1,2,3])`	偏置 $\boldsymbol{b}$
2 维	矩阵	`torch.randn(3,4)`	权重 $\boldsymbol{W}$
3 维	3 阶张量	`torch.randn(32,28,28)`	一批灰度图
4 维	4 阶张量	`torch.randn(32,3,224,224)`	一批 RGB 图

从数据创建

```python
import torch

# 从Python列表
a = torch.tensor([1, 2, 3])
b = torch.tensor([[1.0, 2.0],
                  [3.0, 4.0]])

# 从NumPy数组（共享内存！）
import numpy as np
arr = np.array([1, 2, 3])
t = torch.from_numpy(arr)
```

用函数创建

```python
# 全零/全一
z = torch.zeros(3, 4)
o = torch.ones(2, 3)
# 随机数
r = torch.randn(3, 4)  # N(0,1)
u = torch.rand(3, 4)   # [0,1)
# 等间距序列
s = torch.arange(0, 10, 2)  # [0,2,4,6,8]
# 单位矩阵
I = torch.eye(3)
# 类似已有Tensor的形状
x = torch.randn(2, 3)
y = torch.zeros_like(x)
```

注意

`torch.from_numpy` 与原数组**共享内存**——修改一个，另一个也变！

```python
x = torch.randn(3, 4)
print(x.shape)      # torch.Size([3, 4]) — 形状
print(x.dtype)      # torch.float32      — 数据类型
print(x.device)     # cpu                — 所在设备
print(x.ndim)       # 2                  — 维度数
print(x.numel())    # 12                 — 元素总数
```

常用数据类型

类型	说明	典型用途
torch.float32	32 位浮点（**默认**）	模型参数、中间计算
torch.float16	16 位浮点	混合精度训练
torch.int64	64 位整数（long）	类别标签、索引
torch.bool	布尔值	掩码、条件选择

常见错误

标签必须是 long 类型！`labels = torch.tensor([0, 2, 1], dtype=torch.long)`

逐元素运算

```
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
c = a + b           # [5, 7, 9]
d = a * b           # [4, 10, 18]
e = a ** 2          # [1, 4, 9]
# 数学函数
f = torch.exp(a)    # e^a
g = torch.log(a)    # ln(a)
h = torch.clamp(a, 0, 2)  # [1, 2, 2]
```

矩阵运算

```
A = torch.randn(3, 4)
B = torch.randn(4, 5)
# 矩阵乘法（三种等价写法）
C = A @ B               # 推荐!
C = torch.matmul(A, B)
C = torch.mm(A, B)
# 转置
D = A.T  # 或 A.t()
# 批量矩阵乘法
X = torch.randn(32, 3, 4)
Y = torch.randn(32, 4, 5)
Z = X @ Y  # (32, 3, 5)
```

对应第 2 讲数学

`A @ B` 就是矩阵乘法 $C_{ij} = \sum_k A_{ik} B_{kj}$，`X @ Y` 对应批量前向传播。

聚合运算

```
x = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])
x.sum()          # 21.0   全局求和
x.mean()         # 3.5    全局均值
# 沿指定维度聚合
x.sum(dim=0)     # [5, 7, 9]   按列求和
x.sum(dim=1)     # [6, 15]     按行求和
x.mean(dim=1)    # [2, 5]      按行均值
# argmax：返回最大值的索引
x.argmax(dim=1) # [2, 2]
```

广播（Broadcasting）

```
# 不同形状的Tensor自动广播
A = torch.randn(3, 4)  # (3,4)
b = torch.randn(4)     # (4,)
C = A + b  # b自动扩展为(3,4)
```

广播规则（右对齐比较）：

- 维度相同 → OK
- 某个维度为 1 → 自动扩展
- 缺少维度 → 左边补 1

对应前向传播

$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{a} + \boldsymbol{b}$ 中，$\boldsymbol{b}$ 就是通过广播加到每个样本上的！

```
x = torch.randn(2, 3, 4)  # (2, 3, 4)
# reshape: 改变形状（元素总数不变）
y = x.reshape(2, 12)      # (2, 12)
z = x.reshape(-1, 4)      # (6, 4)  -1自动推断
# view: 与reshape类似，要求内存连续
y = x.view(2, 12)
# squeeze / unsqueeze: 去掉/增加维度
a = torch.randn(1, 3, 1)
a.squeeze()       # (3,)      去掉所有大小为1的维度
a.unsqueeze(0)    # (1, 1, 3, 1)  在第0维增加一维
# flatten: 展平（CNN到全连接的关键操作）
img = torch.randn(32, 1, 28, 28)  # 一批图像
flat = img.flatten(1)              # (32, 784)  保留batch维
```

重要：`flatten(1)` 是 CNN 接全连接层的必备操作

将 (B, C, H, W) 变为 $(B, C \times H \times W)$，保留 batch 维度，后面全部展平。

GPU 有数千个核心，矩阵运算可获得 10∼100 倍加速。

```
# 检查GPU是否可用
print(torch.cuda.is_available())  # True/False
# 设置设备（通用写法，无GPU自动回退CPU）
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
# 将Tensor移到GPU
x = torch.randn(1000, 1000)
x_gpu = x.to(device)            # 移到GPU
# 将模型移到GPU
# model = model.to(device)
# GPU上的Tensor转回CPU（绘图/保存时需要）
x_cpu = x_gpu.cpu()
x_numpy = x_gpu.cpu().numpy()  # 转NumPy必须先回CPU
```

黄金法则

模型和数据必须在同一设备上！一个在 GPU 一个在 CPU 会报错。

操作	NumPy	PyTorch
创建	`np.array([1,2,3])`	`torch.tensor([1,2,3])`
全零	`np.zeros((3,4))`	`torch.zeros(3,4)`
随机	`np.random.randn(3,4)`	`torch.randn(3,4)`
形状	`a.shape`	`a.shape`
变形	`a.reshape(2,6)`	`a.reshape(2,6)`
矩阵乘	`A @ B`	`A @ B`
求和	`a.sum(axis=0)`	`a.sum(dim=0)`
最大值索引	`a.argmax(axis=1)`	`a.argmax(dim=1)`
类型转换	`a.astype(np.float32)`	`a.float()`
GPU 支持	✗	✓ `a.to('cuda')`
自动求导	✗	✓ `requires_grad=True`

记忆要点

PyTorch 的 API 与 NumPy **95% 相同**。额外超能力：**GPU** 和**自动求导**。

回顾第 3 讲：训练需要计算梯度

$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} L$，需要对**每个参数**计算 $\frac{\partial L}{\partial \theta}$。

手动计算（第 3 讲）

- 对每一层推导梯度公式
- 逐个实现反向传播
- 修改网络 → 重新推导
- 容易出错，难以维护

自动求导（PyTorch）

- 只写**前向传播**代码
- 调用 `loss.backward()`
- 自动计算**所有梯度**
- 修改网络 → **自动适应**

核心思想

autograd 在前向传播时自动构建**计算图**，然后利用**链式法则**反向计算梯度——正是第 3 讲的反向传播！

最简单的例子：$y = x^2$，求 $\left.\frac{dy}{dx}\right|_{x=3}$

```python
import torch
# 1. 创建Tensor，开启梯度追踪
x = torch.tensor(3.0, requires_grad=True)
# 2. 前向计算（PyTorch自动记录计算图）
y = x ** 2  # y = 9.0
# 3. 反向传播：计算梯度
y.backward()
# 4. 读取梯度
print(x.grad)  # tensor(6.0)  — 因为 dy/dx = 2x = 6
```

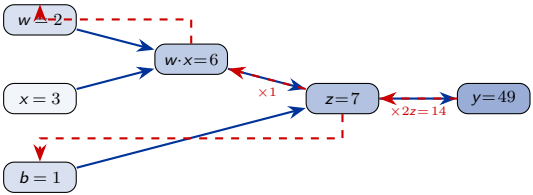
发生了什么?

1. `requires_grad=True`：请记录对 `x` 的所有运算
2. `y = x ** 2`：前向计算 + 后台构建计算图
3. `y.backward()`：沿计算图反向传播，算出 $\frac{dy}{dx}$
4. 梯度存储在 `x.grad` 中

代码

```
w = torch.tensor(2.0,
    requires_grad=True)
b = torch.tensor(1.0,
    requires_grad=True)
x = torch.tensor(3.0)

z = w * x + b    # z = 7.0
y = z ** 2        # y = 49.0
y.backward()
print(w.grad)  # 42.0
print(b.grad)  # 14.0
```



$$\frac{\partial y}{\partial w} = 2z \times x = 14 \times 3 = 42$$
$$\frac{\partial y}{\partial b} = 2z \times 1 = 14$$

模拟两层网络

```python
# 模拟两层网络：输入x=1，目标y=0
x = torch.tensor(1.0)
y = torch.tensor(0.0)
w1 = torch.tensor(0.5, requires_grad=True)
w2 = torch.tensor(-0.3, requires_grad=True)
# 前向传播
h = torch.relu(w1 * x)    # 隐藏层：ReLU(0.5) = 0.5
pred = w2 * h              # 输出层：-0.3 * 0.5 = -0.15
loss = (pred - y) ** 2     # MSE: (-0.15)^2 = 0.0225
loss.backward()
print(f"w1.grad = {w1.grad:.4f}")  # -0.0900
print(f"w2.grad = {w2.grad:.4f}")  # -0.1500
```

验证链式法则

$$\frac{\partial L}{\partial w_2} = 2(\hat{y} - y) \cdot h = 2(-0.15) \times 0.5 = -0.15 \checkmark$$

$$\frac{\partial L}{\partial w_1} = 2(\hat{y} - y) \cdot w_2 \cdot \mathrm{ReLU}'(w_1 x) \cdot x = 2(-0.15) \times (-0.3) \times 1 \times 1 = 0.09$$

但 PyTorch 给出 -0.09，因为 $\hat{y} - y = -0.15$，梯度方向是让 w_1 增大 ✓

细节 1：梯度会累加

```
x = torch.tensor(2.0, requires_grad=True)
y1 = x ** 2
y1.backward()
print(x.grad)  # 4.0
y2 = x ** 3
y2.backward()
print(x.grad)  # 4.0+12.0=16.0 累加了!
# 必须手动清零
x.grad.zero_()
```

训练循环中必须每次清零梯度！

细节 2：torch.no_grad()

```
# 推理时不需要梯度，节省内存
with torch.no_grad():
    pred = model(x_test)
```

细节 3：.item() 取标量

```
# 记录loss数值（脱离计算图）
loss_val = loss.item()  # Python float
losses.append(loss.item())
```

问题：拟合 $y = 3x + 2$，只用 Tensor 和 autograd

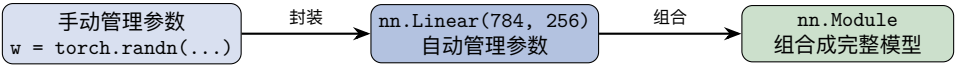
```
torch.manual_seed(42)
X = torch.randn(100, 1)
y = 3 * X + 2 + 0.1 * torch.randn(100, 1)  # y = 3x + 2 + 噪声
w = torch.randn(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
lr = 0.1
for epoch in range(100):
    pred = X * w + b                        # 前向传播
    loss = ((pred - y) ** 2).mean()         # MSE损失
    loss.backward()                         # 反向传播
    with torch.no_grad():                   # 参数更新（不被autograd记录）
        w -= lr * w.grad
        b -= lr * b.grad
    w.grad.zero_()                          # 清零梯度（极其重要！）
    b.grad.zero_()
# 最终结果: w -> 3.0, b -> 2.0
```

训练循环三步曲

1. 前向传播 + 计算损失 → **2.** `loss.backward()` → **3.** 更新参数 + 清零梯度

手动写网络的问题

刚才的例子中我们手动管理 w 和 b。真实网络有成千上万参数——手动管理不现实。



torch.nn 提供的核心组件

- **层**：nn.Linear、nn.Conv2d、nn.BatchNorm1d 等
- **激活函数**：nn.ReLU、nn.Sigmoid 等
- **损失函数**：nn.CrossEntropyLoss、nn.MSELoss 等
- **容器**：nn.Sequential、nn.ModuleList 等

回顾：$z = Wx + b, \quad W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$

```python
import torch.nn as nn
# 创建一个全连接层：输入784维，输出256维
layer = nn.Linear(in_features=784, out_features=256)
# 查看参数
print(layer.weight.shape)  # torch.Size([256, 784])
print(layer.bias.shape)    # torch.Size([256])
# 前向传播
x = torch.randn(32, 784)  # 一批32个样本，每个784维
y = layer(x)               # 输出：(32, 256)
```

nn.Linear 帮你做了什么？

- 自动创建权重 $\boldsymbol{W}$ 和偏置 $\boldsymbol{b}$（默认 Kaiming 初始化）
- 自动设置 requires_grad=True
- 调用时自动计算 $\boldsymbol{x}\boldsymbol{W}^\top + \boldsymbol{b}$

常用层

```
nn.Linear(784, 256)        # 全连接层
nn.BatchNorm1d(256)        # Batch Norm
nn.Dropout(p=0.5)          # Dropout
nn.Flatten()               # 展平
```

激活函数

```
nn.ReLU()        # 隐藏层默认
nn.Sigmoid()     # 二分类输出层
```

重要：CrossEntropyLoss

nn.CrossEntropyLoss = **Softmax + 交叉熵**

输入是**原始 logits**，不要手动加 Softmax！

```
criterion = nn.CrossEntropyLoss()
# 输入: logits (B, C) float
# 标签: (B,) long类型
loss = criterion(logits, labels)
```

```
model = nn.Sequential(
    nn.Linear(784, 256),      # 第1层：784 -> 256
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Dropout(0.5),

    nn.Linear(256, 128),      # 第2层：256 -> 128
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Dropout(0.5),

    nn.Linear(128, 10),       # 输出层：128 -> 10（10个类别）
)
x = torch.randn(32, 784)
logits = model(x)  # (32, 10)
```

对应第 4 讲的标准配置

每个隐藏层：Linear → BatchNorm → ReLU → Dropout——就是第 4 讲的**现代标准配置**！

```python
class MLP(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=256, num_classes=10):
        super().__init__()                # 必须调用父类构造函数
        self.layer1 = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim), nn.ReLU(), nn.Dropout(0.5))
        self.layer2 = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.BatchNorm1d(hidden_dim // 2), nn.ReLU(), nn.Dropout(0.5))
        self.classifier = nn.Linear(hidden_dim // 2, num_classes)

    def forward(self, x):                # 定义前向传播
        x = self.layer1(x)
        x = self.layer2(x)
        return self.classifier(x)
```

```python
model = MLP(input_dim=784, hidden_dim=256, num_classes=10)
total_params = sum(p.numel() for p in model.parameters())
print(f"总参数量: {total_params:,}")       # 235,146
```

最容易踩的坑：BatchNorm 和 Dropout 在训练/推理时行为不同

model.train() —训练模式

- Dropout：随机丢弃神经元
- BatchNorm：用 batch 统计量

model.eval() —评估模式

- Dropout：**关闭**
- BatchNorm：用**全局统计量**

场景	模式	命令
训练	train	`model.train()`
验证	eval	`model.eval() + torch.no_grad()`
测试	eval	`model.eval() + torch.no_grad()`

忘记切换的后果

忘记 `model.eval()` → 验证时 Dropout 仍在丢弃 → 验证精度偏低 → **误判模型性能**！

```python
import torch.optim as optim
# Adam优化器（第4讲推荐的默认选择）
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
# SGD + Momentum（经典选择）
# optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

训练循环中的五行核心

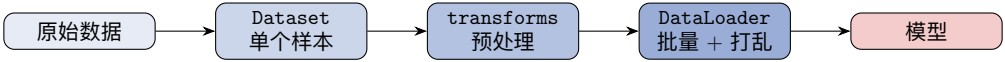
```python
for x_batch, y_batch in dataloader:
    logits = model(x_batch)            # 1. 前向传播
    loss = criterion(logits, y_batch)  # 2. 计算损失
    loss.backward()                    # 3. 反向传播
    optimizer.step()                   # 4. 更新参数
    optimizer.zero_grad()              # 5. 清零梯度
```

学习率调度器

```python
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs, eta_min=1e-6)
# 每个epoch结束后调用：scheduler.step()
```

实际训练中的数据需求

数据量大不能一次加载 / 需要分批训练（Mini-batch） / 需要预处理和打乱



两大组件

- `Dataset`：定义如何获取**一个**样本
- `DataLoader`：定义如何**分批**、**打乱**、**并行**加载

MNIST 升级版：10 类服饰图像，28×28 灰度图，训练 60K/测试 10K

```
import torchvision
import torchvision.transforms as transforms
# 定义数据预处理
transform = transforms.Compose([
    transforms.ToTensor(),                  # PIL图像 -> Tensor，归一化到[0,1]
    transforms.Normalize((0.5,), (0.5,))    # 标准化到[-1, 1]
])
# 下载并加载数据集
train_dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=False, download=True, transform=transform)
print(f"训练集：{len(train_dataset)}，测试集：{len(test_dataset)}")  # 60000, 10000
# 查看单个样本
image, label = train_dataset[0]
print(f"图像形状：{image.shape}，标签：{label}")  # [1, 28, 28], 9
```

10 个类别

T 恤、裤子、套衫、裙子、外套、凉鞋、衬衫、运动鞋、包、踝靴

```
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(),      # 随机水平翻转（数据增强）
    transforms.RandomRotation(10),          # 随机旋转±10度
    transforms.ToTensor(),                  # 转为Tensor + [0,1]归一化
    transforms.Normalize((0.5,), (0.5,))    # 标准化到[-1,1]
])
transform_test = transforms.Compose([
    transforms.ToTensor(),                  # 测试集不做数据增强!
    transforms.Normalize((0.5,), (0.5,))
])
```

重要原则

数据增强只用于训练集，测试/验证集只做必要的预处理。

Normalize 的含义

Normalize(mean, std)：$x_{\text{norm}} = \frac{x - \text{mean}}{\text{std}}$。(0.5,), (0.5,) $\rightarrow \frac{x-0.5}{0.5} \rightarrow$ 将 $[0, 1]$ 映射到 $[-1, 1]$

```python
from torch.utils.data import DataLoader
train_loader = DataLoader(train_dataset,
    batch_size=64, shuffle=True, num_workers=2, pin_memory=True)
test_loader = DataLoader(test_dataset,
    batch_size=256, shuffle=False, num_workers=2)
```

DataLoader 做了什么？

- 自动分批（batching）
- 自动打乱（shuffling）
- 多进程并行加载
- 合并为批次 Tensor

迭代 DataLoader

```python
for images, labels in train_loader:
    print(images.shape)  # (64,1,28,28)
    print(labels.shape)  # (64,)
    break
```

训练集训练模型，验证集调超参数，测试集最终评估（只用一次）

```
from torch.utils.data import random_split
# 从训练集中划出10%作为验证集
train_size = int(0.9 * len(train_dataset))  # 54000
val_size = len(train_dataset) - train_size   # 6000
train_subset, val_subset = random_split(train_dataset, [train_size, val_size],
    generator=torch.Generator().manual_seed(42))
train_loader = DataLoader(train_subset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_subset, batch_size=256, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=256, shuffle=False)
```

```python
from torch.utils.data import Dataset
class MyDataset(Dataset):
    def __init__(self, data, labels, transform=None):
        self.data = data
        self.labels = labels
        self.transform = transform
    def __len__(self):                # 返回数据集大小
        return len(self.data)
    def __getitem__(self, idx):       # 返回第idx个样本
        x, y = self.data[idx], self.labels[idx]
        if self.transform:
            x = self.transform(x)
        return x, y
# 使用方式完全一样
dataset = MyDataset(X_data, y_labels)
loader = DataLoader(dataset, batch_size=64, shuffle=True)
```

三个方法

`__init__`：准备数据　`__len__`：多少个样本　`__getitem__`：取第 i 个

任务描述

- **输入**：28×28 灰度服饰图像 $\rightarrow$ **输出**：10 类中的一类
- **方法**：MLP，应用第 4 讲所有训练技巧 $\rightarrow$ **目标**：准确率 $> 88\%$

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split
import torchvision
import torchvision.transforms as transforms

BATCH_SIZE, EPOCHS, LR = 64, 30, 1e-3
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))  # FashionMNIST全局统计量
])
full_train = torchvision.datasets.FashionMNIST('./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.FashionMNIST('./data', train=False, transform=transform)
train_dataset, val_dataset = random_split(full_train, [54000, 6000],
    generator=torch.Generator().manual_seed(42))
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=256, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=256, shuffle=False)
```

完整项目实战：FashionMNIST 分类 | Step 2：模型定义

```python
class FashionMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()                          # (B, 1, 28, 28) -> (B, 784)
        self.network = nn.Sequential(
            nn.Linear(784, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, 10)  # 不加Softmax! CrossEntropyLoss内含Softmax
        )
    def forward(self, x):
        return self.network(self.flatten(x))
```

```python
model = FashionMLP().to(DEVICE)
print(f"参数总量: {sum(p.numel() for p in model.parameters()):,}")  # ~570K
# 维度检查
dummy = torch.randn(2, 1, 28, 28).to(DEVICE)
print(f"输入: {dummy.shape} -> 输出: {model(dummy).shape}")  # [2,1,28,28] -> [2,10]
```

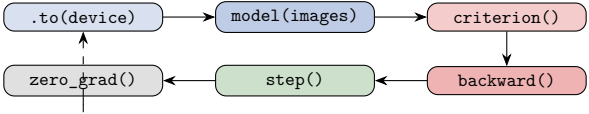
```
criterion = nn.CrossEntropyLoss()                                    # 多分类 -> 交叉熵
optimizer = optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)  # Adam + L2正则化
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS, eta_min=1e-6)
```

第 4 讲理论 → 代码的对应

配置项	第 4 讲理论	代码实现
初始化	He 初始化	`nn.Linear` 默认
归一化	BatchNorm	`nn.BatchNorm1d`
激活	ReLU	`nn.ReLU()`
正则化	Dropout + L2	`Dropout(0.3)` + `weight_decay`
损失	交叉熵	`CrossEntropyLoss()`
优化器	Adam	`Adam(lr=1e-3)`
调度	余弦退火	`CosineAnnealingLR`

完整项目实战：FashionMNIST 分类 | Step 4：训练函数

```python
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()                                   # 训练模式
    total_loss, correct, total = 0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        # ===== 训练五行核心 =====
        logits = model(images)                      # 1. 前向传播
        loss = criterion(logits, labels)            # 2. 计算损失
        loss.backward()                             # 3. 反向传播
        optimizer.step()                            # 4. 更新参数
        optimizer.zero_grad()                       # 5. 清零梯度
        # 统计
        total_loss += loss.item() * labels.size(0)
        correct += (logits.argmax(1) == labels).sum().item()
        total += labels.size(0)
    return total_loss / total, correct / total
```



每一步对应的理论

- `model(images)`：$\boldsymbol{a}^{(l)} = f(\boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)})$（第 3 讲前向传播）
- `criterion(logits, labels)`：$L = -\log \hat{y}_c$（第 4 讲交叉熵）
- `loss.backward()`：链式法则 $\frac{\partial L}{\partial \boldsymbol{W}^{(l)}}$（第 3 讲反向传播）
- `optimizer.step()`：$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{\hat{v}}+\epsilon}\hat{m}$（第 4 讲 Adam）
- `zero_grad()`：清零梯度防止累加（本讲 autograd 细节）

```python
@torch.no_grad()  # 不计算梯度，节省内存
def evaluate(model, loader, criterion, device):
    model.eval()                                        # 评估模式
    total_loss, correct, total = 0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        loss = criterion(logits, labels)
        total_loss += loss.item() * labels.size(0)
        correct += (logits.argmax(1) == labels).sum().item()
        total += labels.size(0)
    return total_loss / total, correct / total
```

验证与训练的两个关键区别

1. `model.eval()` → 关闭 Dropout、BN 切换模式
2. `@torch.no_grad()` → 不记录计算图，节省内存

```python
history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
best_val_acc = 0

for epoch in range(EPOCHS):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, DEVICE)
    val_loss, val_acc = evaluate(model, val_loader, criterion, DEVICE)
    scheduler.step()  # 更新学习率

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

    if val_acc > best_val_acc:  # 保存最优模型
        best_val_acc = val_acc
        torch.save(model.state_dict(), 'best_model.pth')

    print(f"Epoch [{epoch+1:2d}/{EPOCHS}] "
          f"Train: {train_loss:.4f}/{train_acc:.4f} | "
          f"Val: {val_loss:.4f}/{val_acc:.4f} | "
          f"LR: {scheduler.get_last_lr()[0]:.6f}")
```

典型训练日志

```
Epoch [ 1/30] Train: 0.5832/0.7891 | Val: 0.4201/0.8502 | LR: 0.000994
Epoch [ 5/30] Train: 0.3418/0.8741 | Val: 0.3285/0.8805 | LR: 0.000878
Epoch [10/30] Train: 0.2965/0.8904 | Val: 0.3012/0.8917 | LR: 0.000655
Epoch [20/30] Train: 0.2502/0.9072 | Val: 0.2885/0.8973 | LR: 0.000190
Epoch [30/30] Train: 0.2341/0.9131 | Val: 0.2842/0.8992 | LR: 0.000001
```

观察训练过程

- 训练损失**持续下降**：模型在不断学习 ✓
- 验证损失也在下降但较慢：模型在泛化 ✓
- 训练准确率 $>$ 验证准确率：轻微过拟合，差距可控 ✓
- 学习率从 10^{-3} 平滑降到 10^{-6}：余弦退火 ✓

```python
# 加载最优模型
model.load_state_dict(torch.load('best_model.pth'))
test_loss, test_acc = evaluate(model, test_loader, criterion, DEVICE)
print(f"测试准确率: {test_acc:.4f}")  # 约 0.89+
```

分类别准确率

```python
class_correct = [0] * 10
class_total = [0] * 10
model.eval()
with torch.no_grad():
  for images, labels in test_loader:
    images = images.to(DEVICE)
    pred = model(images).argmax(1).cpu()
    for i in range(len(labels)):
      label = labels[i].item()
      class_total[label] += 1
      if pred[i] == label:
        class_correct[label] += 1
```

典型结果

类别	Acc	类别	Acc
T 恤	.856	凉鞋	.975
裤子	.976	衬衫	.723
套衫	.821	运动鞋	.957
裙子	.912	包	.968
外套	.845	踝靴	.961

衬衫最难——易与 T 恤、外套混淆。

```
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(history['train_loss'], label='Train'); ax1.plot(history['val_loss'], label='Val', ls='--')
ax1.set_xlabel('Epoch'); ax1.set_ylabel('Loss'); ax1.legend()
ax2.plot(history['train_acc'], label='Train'); ax2.plot(history['val_acc'], label='Val', ls='--')
ax2.set_xlabel('Epoch'); ax2.set_ylabel('Accuracy'); ax2.legend()
plt.tight_layout(); plt.show()
```

诊断指南（回顾第 4 讲）

现象	诊断	处方
训练/验证损失都高	欠拟合	增大网络、提高 lr
训练低、验证高	过拟合	加 Dropout/L2、数据增强
验证损失上升	过拟合开始	Early Stopping

训练阶段

1. 忘记 `zero_grad()` → NaN
2. 忘记 `model.train()`
3. 数据没移到 GPU → 报错
4. 标签类型非 `long`
5. 学习率太大 → 震荡

评估阶段

6. 忘记 `model.eval()`
7. 忘记 `no_grad()` → OOM
8. 多加了 Softmax → 重复
9. 维度不匹配
10. 数据泄露（测试集调参）

```python
class DebugMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)
    def forward(self, x):
        print(f"输入: {x.shape}")            # [B, 1, 28, 28]
        x = self.flatten(x); print(f"展平后: {x.shape}")   # [B, 784]
        x = torch.relu(self.fc1(x)); print(f"第1层后: {x.shape}")  # [B, 256]
        x = torch.relu(self.fc2(x)); print(f"第2层后: {x.shape}")  # [B, 128]
        x = self.fc3(x); print(f"输出: {x.shape}")          # [B, 10]
        return x
model = DebugMLP(); model(torch.randn(2, 1, 28, 28))  # 测试维度
```

养成好习惯

在正式训练前用 2 个样本测试前向传播，确保维度正确。维度错误是最常见的 bug！

```python
model = FashionMLP().to(DEVICE)
images, labels = next(iter(train_loader))
images, labels = images.to(DEVICE), labels.to(DEVICE)
loss = criterion(model(images), labels)
loss.backward()
for name, param in model.named_parameters():
    if param.grad is not None:
        g = param.grad
        print(f"{name:30s} | mean: {g.mean():.6f} | std: {g.std():.6f} | max: {g.abs().max():.6f}")
```

梯度异常信号

- 梯度全为 0 → 学习率为 0 / ReLU 死亡 / 零初始化
- 梯度极大（> 100）→ 梯度爆炸，需裁剪或降 lr
- 梯度极小（$< 10^{-7}$）→ 梯度消失，检查激活函数

先让模型在极小数据上过拟合——拟合不了说明代码有 bug

```
tiny_dataset = torch.utils.data.Subset(train_dataset, range(100))
tiny_loader = DataLoader(tiny_dataset, batch_size=100, shuffle=True)
model_debug = FashionMLP().to(DEVICE)
optimizer_debug = optim.Adam(model_debug.parameters(), lr=1e-2)
model_debug.train()
for epoch in range(200):
    for images, labels in tiny_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        loss = criterion(model_debug(images), labels)
        loss.backward(); optimizer_debug.step(); optimizer_debug.zero_grad()
    if epoch % 50 == 0:
        print(f"Epoch {epoch}: Loss = {loss.item():.6f}")
# 期望: loss降到接近0
```

核心原则

如果 loss 降不下去 → 检查代码 bug，而不是调超参数！

```python
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter('runs/fashion_mlp')
for epoch in range(EPOCHS):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, DEVICE)
    val_loss, val_acc = evaluate(model, val_loader, criterion, DEVICE)
    writer.add_scalars('Loss', {'train': train_loss, 'val': val_loss}, epoch)
    writer.add_scalars('Accuracy', {'train': train_acc, 'val': val_acc}, epoch)
    writer.add_scalar('LR', scheduler.get_last_lr()[0], epoch)
    scheduler.step()
writer.close()
# 终端运行: tensorboard --logdir=runs
# 浏览器打开: http://localhost:6006
```

TensorBoard 的优势

实时监控训练过程，对比不同实验，可视化 loss/accuracy/lr 曲线。

方式一：只保存参数（推荐）

```
# 保存
torch.save(model.state_dict(), 'weights.pth')
# 加载
model = FashionMLP()
model.load_state_dict(torch.load('weights.pth'))
model.eval()
```

方式二：保存整个模型

```
# 保存
torch.save(model, 'model_full.pth')
# 加载
model = torch.load('model_full.pth')
model.eval()
```

缺点：依赖类定义、文件大

保存完整检查点（用于恢复训练）

```
checkpoint = {'epoch': epoch, 'model_state_dict': model.state_dict(),
              'optimizer_state_dict': optimizer.state_dict(), 'best_val_acc': best_val_acc}
torch.save(checkpoint, f'checkpoint_epoch{epoch}.pth')
```

```
checkpoint = torch.load('checkpoint_epoch10.pth')
model = FashionMLP().to(DEVICE)
model.load_state_dict(checkpoint['model_state_dict'])
optimizer = optim.Adam(model.parameters())
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
scheduler.load_state_dict(checkpoint['scheduler_state_dict'])
start_epoch = checkpoint['epoch'] + 1
# 继续训练
for epoch in range(start_epoch, EPOCHS):
    train_one_epoch(model, train_loader, criterion, optimizer, DEVICE)
    ...
```

何时需要恢复?

GPU 被抢占 / 机器重启 / 长时间训练需中断

```python
import random, numpy as np

def set_seed(seed=42):
    """固定所有随机种子"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(42)  # 在所有代码之前调用
```

深度学习中的随机性来源

参数初始化、数据打乱顺序、Dropout、多进程 DataLoader

重要

可复现性是科学实验的基本要求。**论文投稿前务必确认结果可复现。**

```python
import torch, torch.nn as nn, torch.optim as optim
from torch.utils.data import DataLoader, random_split
import torchvision, torchvision.transforms as transforms

BATCH_SIZE, EPOCHS, LR = 64, 30, 1e-3
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ===== 数据 =====
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.2860,), (0.3530,))])
full_train = torchvision.datasets.FashionMNIST('./data', train=True, download=True, transform=transform)
test_data = torchvision.datasets.FashionMNIST('./data', train=False, transform=transform)
train_data, val_data = random_split(full_train, [54000, 6000])
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=256, shuffle=False)
test_loader = DataLoader(test_data, batch_size=256, shuffle=False)

# ===== 模型 =====
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(512, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(256, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, 10)
).to(DEVICE)
```

```python
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
# ===== 训练 =====
for epoch in range(EPOCHS):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        loss = criterion(model(images), labels)
        loss.backward(); optimizer.step(); optimizer.zero_grad()
    scheduler.step()
    # 验证
    model.eval(); correct, total = 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            correct += (model(images).argmax(1) == labels).sum().item()
            total += labels.size(0)
    print(f"Epoch {epoch+1}/{EPOCHS} | Val Acc: {correct/total:.4f}")

# ===== 测试 =====
model.eval(); correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        correct += (model(images).argmax(1) == labels).sum().item()
        total += labels.size(0)
print(f"Test Accuracy: {correct/total:.4f}")
```

第 1–4 讲理论	PyTorch 实现	一行代码
张量与矩阵乘法	Tensor	`A @ B`
前向传播 $\boldsymbol{a} = f(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$	nn.Linear + nn.ReLU	`model(x)`
反向传播（链式法则）	autograd	`loss.backward()`
梯度下降 $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla L$	optim	`optimizer.step()`
交叉熵损失	nn.CrossEntropyLoss	`criterion(logits, y)`
He 初始化	默认	`nn.Linear()`
BatchNorm	nn.BatchNorm1d	`nn.BatchNorm1d(256)`
Dropout	nn.Dropout	`nn.Dropout(0.5)`
L2 正则化	weight_decay	`Adam(..., wd=1e-4)`
余弦退火	lr_scheduler	`CosineAnnealingLR()`

核心结论

PyTorch 将第 1–4 讲的**所有理论**封装成简洁 API。理解原理后，代码只是“翻译”。

今天学了什么

1. **Tensor**：多维数组 + GPU + autograd，API 与 NumPy 几乎一致
2. **autograd**：`requires_grad` → `backward()` → `.grad`
3. **nn.Module**：Sequential 或自定义类，标准配置 = Linear+BN+ReLU+Dropout
4. **Dataset/DataLoader**：数据管道，自动分批、打乱、预处理
5. **完整项目**：FashionMNIST 分类，从数据加载到模型评估
6. **调试技巧**：维度检查、梯度检查、小数据过拟合、TensorBoard
7. **工程实践**：模型保存/加载、检查点恢复、随机种子

训练循环五行核心（必须背！）

`logits = model(x)` → `loss = criterion(logits, y)` → `loss.backward()` → `optimizer.step()` → `optimizer.zero_grad()`

练习 1：修改网络结构

把 3 层 MLP 改成 2 层（784→256→10），对比测试准确率变化。

练习 2：对比 Dropout 效果

分别用 Dropout=0, 0.3, 0.5, 0.8 训练，画四条验证曲线对比。

练习 3：对比优化器

分别用 SGD（lr=0.01, momentum=0.9）和 Adam（lr=1e-3），对比收敛速度。

练习 4（思考题）

为什么 MLP 在 FashionMNIST 上很难超过 90%？什么结构可以更高？
提示：MLP 展平图像为 784 维向量，丢失了**空间结构**信息。→ 下次课 CNN 给出答案。

第 6 讲：深度学习进阶与图像处理准备

- 万能近似定理：为什么神经网络 “理论上能拟合一切”?
- 深度 vs 宽度：为什么深度更高效?
- 新型激活函数：GELU、Swish
- 数据增强技术：Mixup、CutMix
- 全连接处理图像的瓶颈 → CNN 的动机

课后任务

1. 在自己的电脑上跑通今天的完整代码
2. 完成课堂练习 1–3
3. 尝试用 TensorBoard 记录训练过程

谢谢！有问题欢迎随时提问。